SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© Team FsktmHack UMHackathon 2026

1. Introduction:

MediFlow is an AI-assisted medical triage and appointment scheduling web application that addresses inefficiencies in traditional patient intake systems. It enables patients to describe their symptoms in natural language, which are then processed by an AI-powered multi-agent workflow to determine urgency levels, collect missing information, and recommend appropriate actions such as emergency care or appointment booking.

1.1. Purpose:

This system analysis document outlines the technical scope, architecture, and design decisions behind the MediFlow system. It serves as a reference for developers, testers, and stakeholders to understand how the system processes patient input, integrates AI services, and delivers structured medical decisions.

1. *Architecture:*

MediFlow is a cloud-based client-server system consisting of a React frontend, a Node.js backend API, Firestore database, and an AI service layer powered by ilmu-glm-5.1 via a Z.AI-compatible endpoint. The system is deployed using **Render (backend)** and **Vercel (frontend)**.

2. *Data Flows:*

Patient symptom data flows from the frontend interface to the backend API, where it is validated and processed. The backend constructs structured prompts and sends them to the AI service. The AI response is parsed into structured data, stored in Firestore, and returned to the frontend as triage results or follow-up questions.

© Team FsktmHack UMHackathon 2026

3. ***Model Process:***

The MediFlow model represents an end-to-end workflow:

1. Patient submits symptoms
2. AI interprets input
3. Triage classification (Emergency / Urgent / Non-Urgent)
4. Missing information detection
5. Follow-up questioning loop
6. Final decision (Emergency redirect or Appointment scheduling)

4. ***Role of Reference:***

This document serves as a technical reference for system implementation, testing, and evaluation, ensuring all components align with the intended architecture and workflow.

*1.2. **Background**:*

Healthcare systems often rely on manual triage processes, where patients describe symptoms in unstructured formats. This leads to delays, misclassification, and inefficiencies. MediFlow introduces an AI-driven workflow to standardize intake, improve accuracy, and automate decision-making. Traditional systems rely on manual intake handled by healthcare staff, without automation or structured decision support. MediFlow introduces:

- AI-driven triage workflow
- Stateful session management using Firestore
- Automated follow-up questioning
- Deployment using modern cloud platforms (Render & Vercel)
- Integration of LLM (ilmu-glm-5.1) as a service layer

© Team FsktmHack UMHackathon 2026

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
Patient	Input symptoms and receive medical guidance	Fast, accurate triage and recommendations
Healthcare Staff (Doctors/Admin)	Use structured outputs for decision-making	Reduced manual workload and clearer data
Development Team	Build and maintain system	Modular, scalable, well-defined APIs
QA Team	Validate system behavior	Complete test coverage and reliability

2. System Architecture & Design

2.1 High Level Architecture

2.1.1 Overview:

Type	Details
System	Web Application (Browser-based)
Architecture	Client-Server with AI Service Layer

MediFlow consists of:

- Frontend (React Web App)
- Backend (Express API)
- Firestore Database
- AI Service Layer (LLM)

© Team FsktmHack UMHackathon 2026

2.1.2 LLM as Service Layer

MediFlow integrates **ilmu-glm-5.1 via Z.AI API** as a dedicated reasoning engine.

Prompt Flow:

- Backend constructs prompt using:
  - symptoms
  - session state
  - missing fields
- Only relevant context is sent (no full chat history)

Processing:

- AI returns structured JSON:
  - triage level
  - confidence
  - missing fields
  - recommendation

Handling:

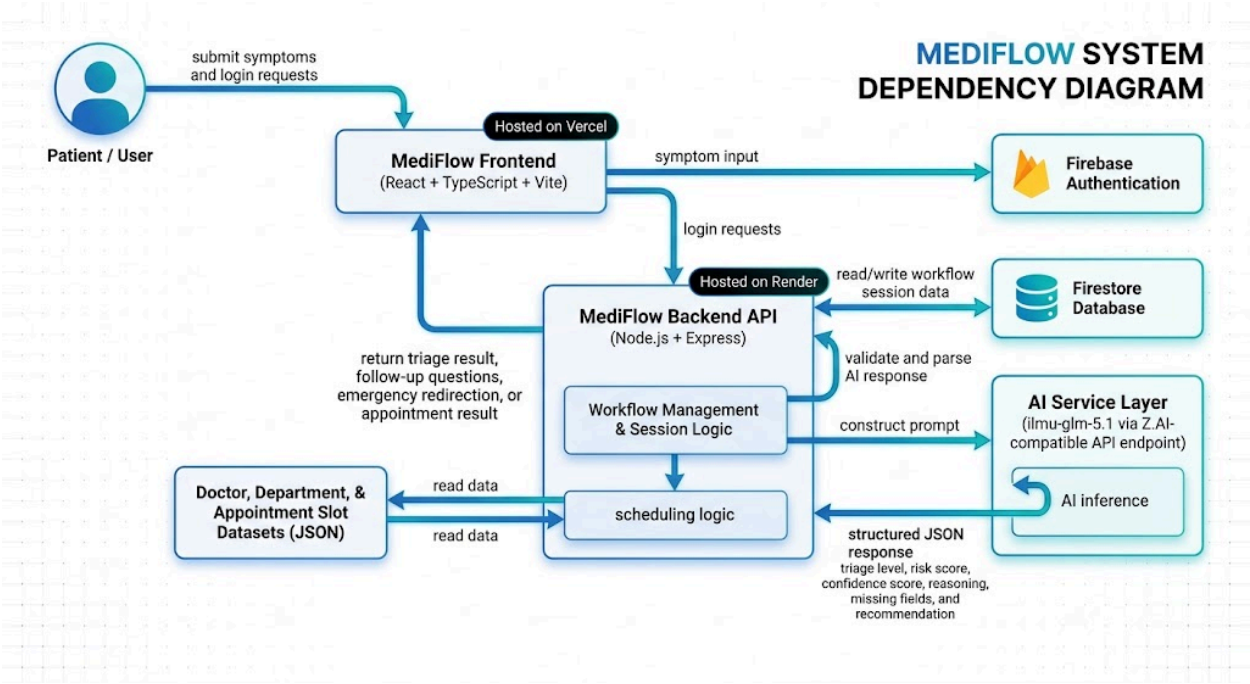
- Backend parses response
- Stores in Firestore
- Sends result to frontend

Optimization:

- Token limits enforced via prompt trimming
- Fallback logic if AI fails
- Rule-based override for critical symptoms

© Team FsktmHack UMHackathon 2026

2.1.3 Dependency Diagram



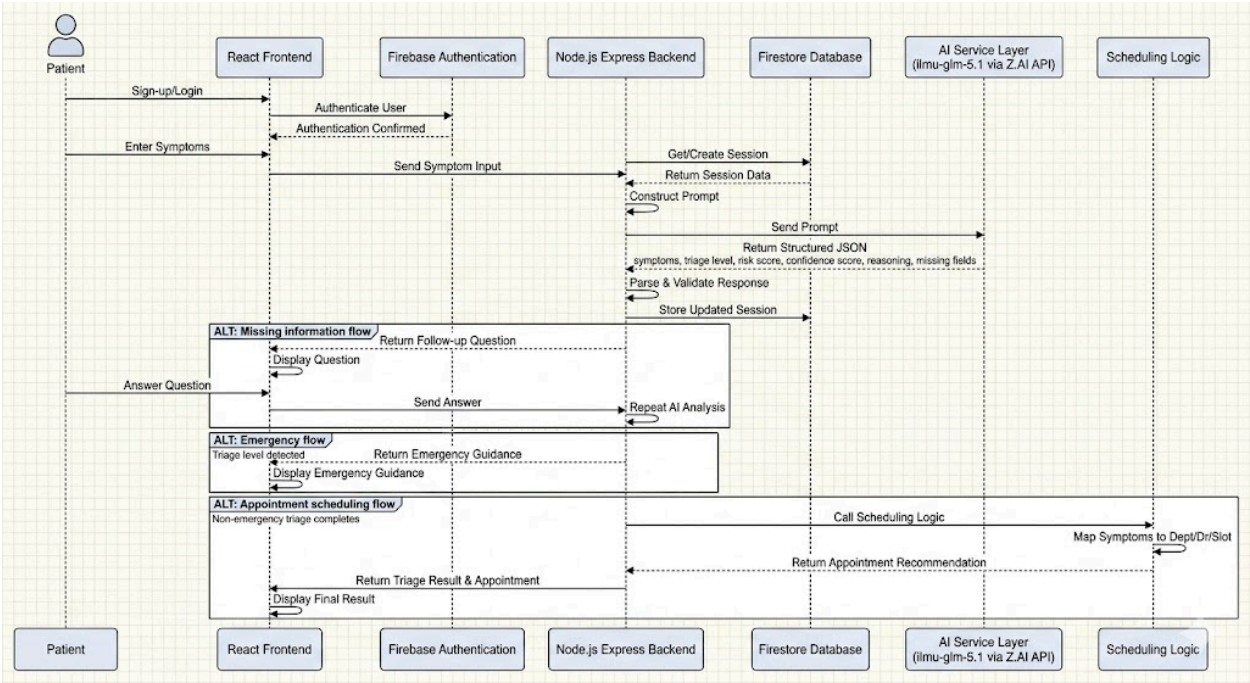
The dependency diagram illustrates how different components of the MediFlow system interact to process patient inputs and generate medical decisions. The process begins when the patient submits symptoms through the frontend interface hosted on Vercel. The frontend communicates with the backend API hosted on Render, which acts as the central controller of the system.

The backend manages workflow sessions using Firestore and constructs structured prompts based on user input and session context. These prompts are sent to the AI service layer powered by ilmu-glm-5.1 via a Z.AI-compatible API. The AI processes the request and returns structured outputs such as triage level, confidence score, reasoning, and missing information.

The backend then validates and parses the AI response, updates the Firestore database, and uses scheduling logic to determine appropriate medical actions. Finally, the processed results are returned to the frontend for display to the user. This architecture ensures clear separation between user interface, backend logic, AI reasoning, and data storage.

© Team FsktmHack UMHackathon 2026

2.1.4 Sequence Diagram



The sequence diagram presents the step-by-step interaction between system components during a typical user session in MediFlow. The process begins when the patient logs in through Firebase Authentication and submits symptom descriptions via the frontend interface.

The frontend sends the input to the backend API, which retrieves or initializes a workflow session from Firestore. The backend then constructs a structured prompt and sends it to the AI service layer for analysis. The AI returns structured outputs including triage classification, confidence level, reasoning, and any missing information.

If additional information is required, the backend generates follow-up questions and sends them to the frontend. This interaction continues iteratively until sufficient data is collected. Once complete, the backend determines whether the case requires emergency redirection or appointment scheduling.

For non-emergency cases, the system assigns a suitable department, doctor, and appointment slot before returning the final decision to the user. This sequence ensures dynamic, stateful, and adaptive interaction throughout the triage process.

© Team FsktmHack UMHackathon 2026

2.4 Technological Stack

2.4.1 Frontend

React, TypeScript, Vite, Tailwind

2.4.2 Backend

Node.js, Express

2.4.3 Database

Firestore

2.4.4 Cloud/Deployment [e.g. AWS EC2 (Server), RDS (PostgreSQL), S3 (Static Assets)

Render (Backend), Vercel (Frontend)

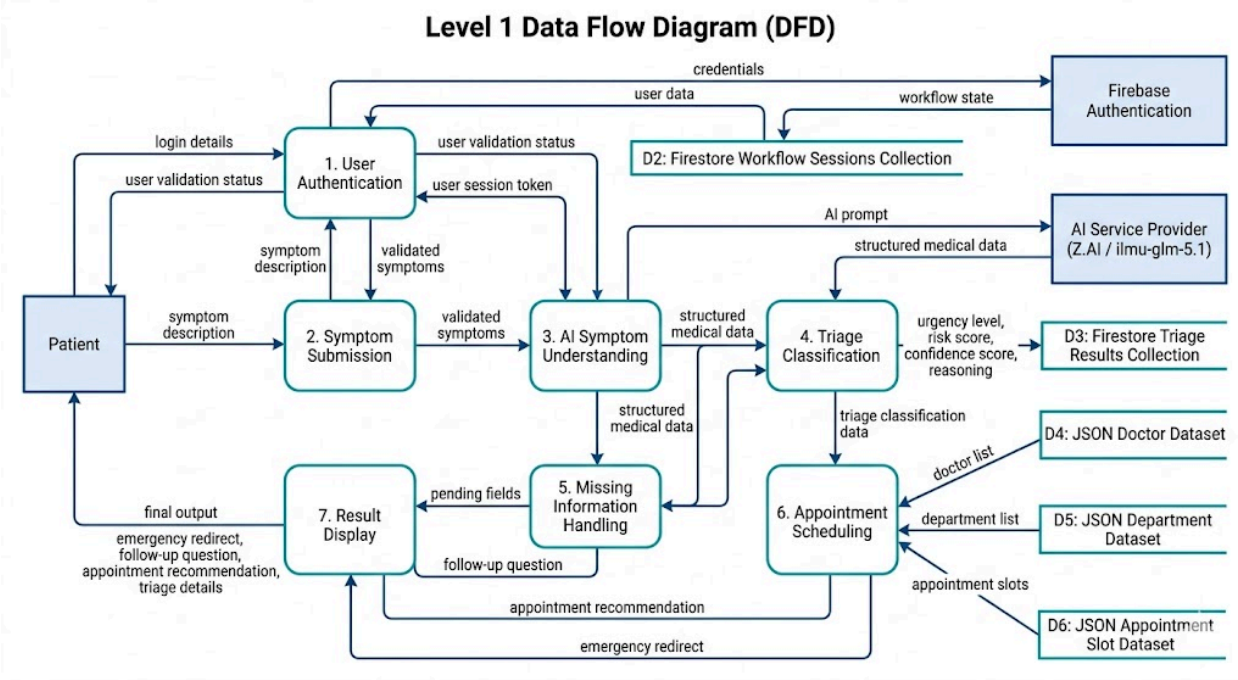
2.4.5 AI

ilmu-glm-5.1 (Z.AI API)

© Team FsktmHack UMHackathon 2026

2.5 Key Data Flows

2.5.1 Data Flow Diagram (DFD)



The Data Flow Diagram (DFD) illustrates how data moves through the MediFlow system across external entities, processes, and data stores. The patient acts as the primary external entity, providing input through the frontend interface.

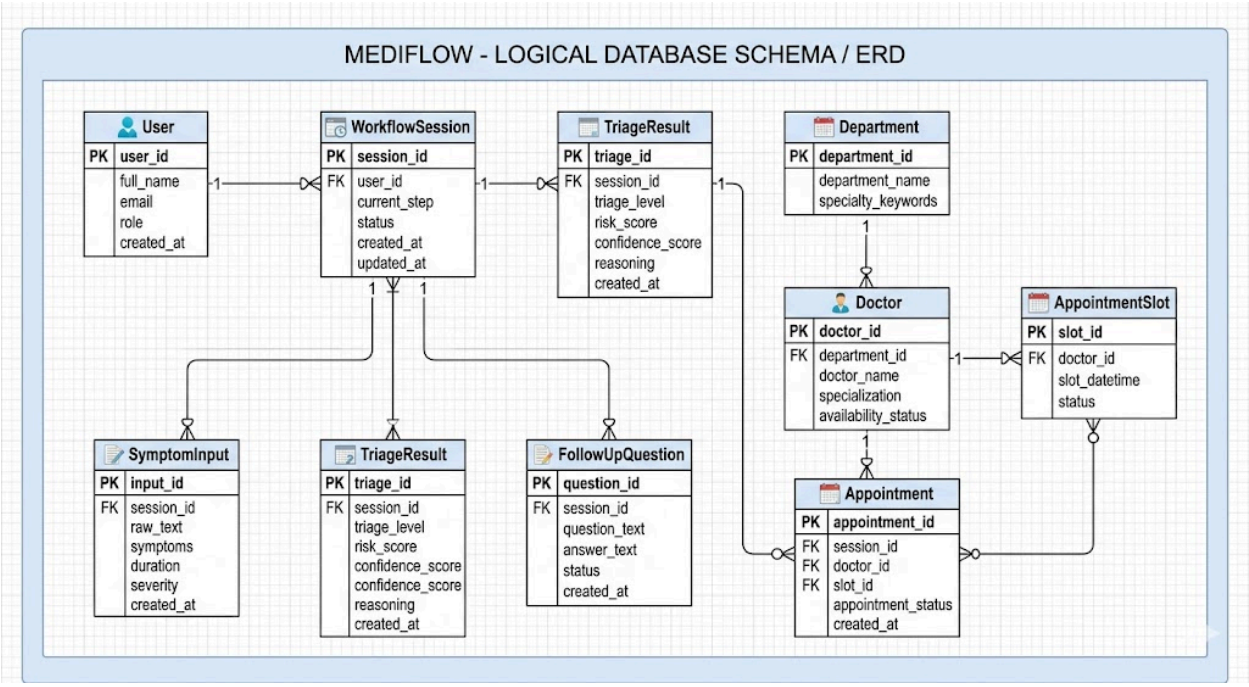
The system processes include user authentication, symptom submission, AI-based symptom understanding, triage classification, missing information handling, and appointment scheduling. These processes transform raw input into structured medical decisions.

Data is stored in Firestore collections, including user data, workflow sessions, and triage results, ensuring persistence and state management. The system also interacts with external AI services via the Z.AI API to perform reasoning and classification.

The DFD ensures that all data inputs are properly processed and stored, avoiding data loss or “black holes.” The final output, whether a follow-up question, emergency redirection, or appointment recommendation, is returned to the patient through the frontend interface.

© Team FsktmHack UMHackathon 2026

2.5.2 Normalized Database Schema



The normalized database schema represents the logical data structure of the MediFlow system, ensuring data integrity, consistency, and efficient relationships between entities. Although Firestore is a NoSQL database, the schema is designed conceptually in normalized form for clarity and documentation purposes.

The core entities include User, WorkflowSession, SymptomInput, TriageResult, FollowUpQuestion, Department, Doctor, AppointmentSlot, and Appointment. Each entity is designed with primary keys and appropriate foreign key relationships to maintain referential integrity.

The schema supports one-to-many relationships such as a user having multiple workflow sessions, and a workflow session containing multiple symptom inputs and follow-up questions. It also supports decision-making processes such as mapping symptoms to departments and assigning doctors and appointment slots.

This structured design ensures that the system can efficiently manage patient data, AI outputs, and scheduling logic while maintaining scalability and maintainability.

© Team FsktmHack UMHackathon 2026

3. Functional Requirements & Scope

This section identifies the essential capabilities of MediFlow by considering those functions that would help prove the viability of the MVP. The patients will be able to enter their symptoms using the web interface, and then, depending on the input, the AI multi-agent algorithm will determine the priority of treatment. In cases where further information is needed, MediFlow will automatically generate additional questions for the patient. On the basis of the provided answers, MediFlow will make its decision, which may involve an urgent visit to the doctor or scheduling an appointment.

3.1 Minimum Viable Product:

#	Features	Description
1	Patient Symptom Input	The users report their symptoms via web-based interfaces using natural language processing inputs.
2	AI Triage Classification	The system uses AI technologies for analyzing the symptom reports to classify the urgency of the symptom.
3	Dynamic Follow-up Questions	The system checks for missing information and asks further questions in order to obtain all necessary patient information.
4	Appointment & Action Recommendation	Finally, the system provides recommendations about the course of action that should be taken by the user.
5	Stateful Workflow Management	The system keeps track of the current session to manage multi-step procedures efficiently.

© Team FsktmHack UMHackathon 2026

3.2 Non-Functional Requirements (NFRs)

Quality (e.g.)	**Requirements (e.g.)**	**Implementation (e.g)**
Scalability	MediFlow should support multiple patient sessions at the same time.	Deploy backend on Google Cloud Run to allow automatic scaling based on traffic.
Reliability	The system should store patient sessions properly and avoid losing workflow progress.	Use Firestore to save session state, patient inputs, pending fields, and question history.
Maintainability	The system should be easy to update and debug.	Separate frontend, backend, AI workflow, and database logic into modular components.
Token Latency	AI responses should be returned within a reasonable time to avoid long waiting periods.	Limit prompt size, send only relevant session context, and use timeout handling for AI API calls.
Cost Efficiency	The system should control AI API usage and avoid unnecessary token consumption.	Keep prompts short, avoid repeated full conversation history, and reuse stored session data when possible.

© Team FsktmHack UMHackathon 2026

3.4 Out of Scope / Future Enhancements

- Doctor dashboard
- Real-time chat with healthcare staff
- Integration with hospital systems (EMR)
- Mobile app version
- Multilingual support
- Predictive health analytics

4. Monitor, Evaluation, Assumptions & Dependencies

4.1 Technical Evaluation:

4.1.1 Grayscale Rollout & A/B Testing:

New features are released to a small percentage of users first. Performance is monitored before full rollout. A/B testing is used to optimize AI prompts and workflows.

4.1.2 Emergency Rollback (ER) & Golden Release:

In case of system failure or incorrect AI outputs, the system reverts to a stable version (Golden Release).

4.1.3 Priority Matrix:

Level	Description	Action
P1	Critical failure (AI / API down)	Immediate rollback
P2	High latency or incorrect output	Immediate rollback
P3	Minor bugs	Fix next release
P4	UI improvements	Future update

© Team FsktmHack UMHackathon 2026

4.2 Monitoring:

- API response time
- AI latency
- Error rates
- System availability

Alerts triggered if thresholds exceeded.

4.5 Assumptions:

- Users have stable internet connection
- Users can describe symptoms clearly
- AI API is available
- System operates under moderate load

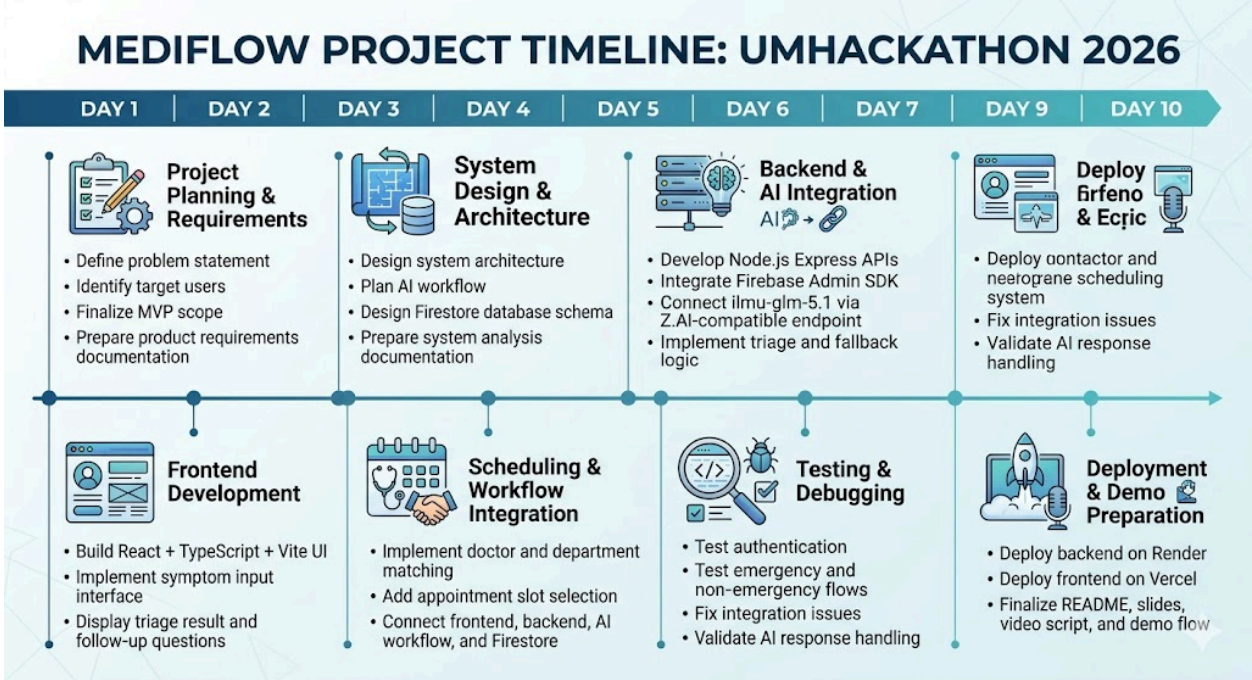
4.6 External Dependencies:

Tools	Purpose	Risks
Z.AI GLM API	AI reasoning engine	High – API limits
Render	Backend hosting	Medium
Vercel	Frontend hosting	Low
Firestore	Data storage	Low
Internet	Communication layer	Medium

5. Project Management & Team Contributions

© Team FsktmHack UMHackathon 2026

5.1 Project Timeline:



The project timeline outlines the development phases of MediFlow over a 10-day hackathon sprint. The process begins with project planning and requirement analysis, where the problem scope and system objectives are defined.

This is followed by system design and architecture planning, including the design of AI workflows and database structure. The development phase focuses on backend implementation, AI integration using ilmu-glm-5.1, and frontend interface development.

Subsequently, system integration is performed to connect all components, including scheduling logic and workflow management. Testing and debugging ensure that all features function correctly and edge cases are handled.

Finally, the system is deployed using Render for the backend and Vercel for the frontend, followed by preparation of presentation materials such as slides, documentation, and demo scripts. This structured timeline ensures efficient delivery of a functional and well-integrated system within the hackathon duration.

© Team FsktmHack UMHackathon 2026

5.2 Team Members Role:

Member	Role	Responsibilities
Mah Kah Mun	Frontend Developer	Developed UI for symptom input, chat interaction, and result display.
Lai Joey	AI Engineer	Built input understanding module to convert user input into structured data.
Yeong Hui Ni	AI Engineer	Implemented AI triage logic for urgency classification and reasoning.
Kwek Chee Ling	Backend Developer	Developed backend APIs, workflow logic, and system integration.
Teoh En Xi	Backend Developer	Implemented scheduling logic and assisted with system documentation.

5.3 Recommendations:

- Implement caching for repeated AI queries
- Add fallback logic for AI failure
- Use monitoring dashboards for real-time insights

© Team FsktmHack UMHackathon 2026